

Dzongkha Next Word Prediction System



Project Report
submitted in partial fulfillment of the requirements for
the award of the degree Bachelor of Engineering
in
Information Technology

Ms. Perna Chhetri (02200180)

Ms. Phuntsho Wangmo (02200160)

Mr. Tenzin Yoezer (02190165)

Mr. Tshewang Bomjan(02200170)

Under the Guidance of:

Mr. Karma Wangchuk

Information Technology Department

College of Science and Technology

Phuentsholing, Bhutan

June, 2024

ROYAL UNIVERSITY OF BHUTAN
COLLEGE OF SCIENCE AND TECHNOLOGY
INFORMATION TECHNOLOGY DEPARTMENT



CERTIFICATE

This is to certify that the B.E. project titled **“Dzongkha Next Word Prediction System”**, which is being submitted by **Ms. Prerna Chettri (02200180)**, **Ms. Phuntsho Wangmo (02200160)**, **Mr. Tenzin Yoezer (02190165)** and **Mr. Tshewang Bomjan (02200170)** in partial fulfillment of the requirement for the award of the degree of Bachelor of Engineering in Information Technology is a record of students work carried out at College of Science and Technology, Phuentsholing under my supervision and guidance.

Mr. Karma Wangchuk

Project Guide

Acknowledgement

We would like to express our sincere gratitude to Mr. Karma Wangchuk, faculty of Information Technology Department, CST, for being our project guide and providing us with the necessary support and guidance that helped shape the quality of our project. We would also like to thank Dzongkha Development Commission (DDC), they provided us with the required number of dataset. The Govtech Agency allowed us to make temporary use of their high-end system for training our model during the OJT period, we also would like to thank them for their support. Lastly, College of Science and Technology provided us with a favorable environment to learn and also provided the support at critical times, We are truly thankful to all the faculties and colleagues.

Abstract

This document showcases the entire process that we followed while carrying out the project. It also mentions some of the software tools and applications that we employed. Our dataset acquired from DCDD has a total of 100000 sentences, 1331282 words and 28344 unique words. The project is mainly aimed to make dzongkha typing as convenient as possible by reducing the number of keystrokes. The AI model is chosen for integration with the front end system based on their performance while training it on a larger datasets. Accuracy is the performance metrics based on which we determine the performance of each model. While observing the results of model training, we initially found out that LSTM performs well when the size of the dataset is small with only 20000 sentences but this gave rise to overfitting. The model was only able to accurate predictions for the samples that were used for training but it was inaccurate with new samples. While training the model on a 100000 sentences dataset, GRU being lightweight and its ability to handle larger dataset performed better with 74.03% accuracy and also solved the problem of overfitting. While developing the application, we designed the user interface (front end) using figma and used HTML, CSS and javascript to develop it and this acts as a foundation of our application's interface. In Order to make our web app responsive, Bootstrap was used. For the backend development, Django, a python based web framework was used as it is convenient for using most of machine learning libraries and handling the server side logic. SQLite is used as a database because of its convenience while integrating with Django.

Terminology

1. **Corpus:** The machine learning models are trained and evaluated by using text and documents that are largely collected. These collections refer to corpus. It serves as the dataset for language pairs and contains source sentences and their corresponding translations.
2. **Training Data:** The dataset consisting of Dzongkha sentences used to train the Next Word prediction model.
3. **Epoch:** Within the realm of machine learning, an epoch means one complete iteration through the entire dataset in the training process. In each epoch, the parameters are adjusted by the model according to the given training examples.
4. **Batch Size:** The amount of data points considered together in a single forward or backward pass during training. A larger batch size takes less time to train, but it requires a lot more memory.
5. **Application Programming Interface:** Communication between different software applications are allowed by the set of protocols and rules called API. In the context of Dzongkha Next word prediction system, the API serves as an interface between the application and the next word prediction service.
6. **GUI:** It stands for Graphical User Interface. The frontend of our system that consists of icons, buttons, and other graphical elements and allows the user to interact with the system

Abbreviations

BERT	Bidirectional Encoder Representations from Transformers
Bi-LSTM	Bidirectional Long Short-Term Memory
CSS	Cascading Style Sheets
DCDD	Department of Culture and Dzongkha Development
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
GloVe	Global Vectors for Word Representation
GPT	Generative Pre-trained Transformer
GUI	Graphical User Interface
HTML	Hypertext Markup Language
LSTM	Long Short-Term Memory
ML	Machine Learning
NLP	Natural Language Processing
RNN	Recurrent Neural Network
TF-IDF	Term Frequency-Inverse Document Frequency
Word2Vec	Word to Vector
XLNet	eXtreme Multilingual Language Model

Table 1: Abbreviations

List of Tables

Table 1. Abbreviations	8
Table 2. 20,000 Dataset	24
Table 3. 100,000 Dataset	25
Table 4. Model Comparison on 20k dataset	38
Table 5. 100K dataset details	51
Table 6. Model comparison on 100k dataset	51

List of Figures

Figure 1: RNN Architecture	16
Figure 2: LSTM model architecture	17
Figure 3: GRU model architecture	18
Figure 4: General Methodology	19
Figure 5: Project Methodology	20
Figure 6: HTML Logo	22
Figure 7: CSS Logo	22
Figure 8: Google Colab Logo	22
Figure 9: JavaScript Logo	22
Figure 10: Bootstrap Logo	22
Figure 11: Django Logo	22
Figure 12: SQLite Logo	22
Figure 13: Data Preprocessing	25
Figure 14: Data before Preprocessing	26
Figure 15: Data after Preprocessing	26
Figure 16: Tokenization process	27
Figure 17: Code implementation for tokenization	27
Figure 18: N gram sequence generation	28
Figure 19: Input sequence before padding	28
Figure 20: Input sequence after padding	29
Figure 21. BERT BASE and BERT LARGE architecture	30
Figure 22. LSTM cell	32

Figure 23. Bi-LSTM model Architecture	35
Figure 24. GRU architecture	36
Figure 25. create_model code snapshot	38
Figure 26. Hyperparameters code snapshot	40
Figure 27. plotting histogram code snapshot	41
Figure 28. Landing page of Dzongkha Nextword Prediction	46
Figure 29. About Us page	46
Figure 30. Exporting Dzongkha Text	47
Figure 31. Importing Dzongkha Text	48
Figure 32. Share functionality	49
Figure 33. Application Tutorial Section	50
Figure 34. LSTM (20K dataset) result in graphical form	52
Figure 35. Bi-LSTM (20K dataset) result in graphical form	53
Figure 36. GRU (20K dataset) results in graphical form	53
Figure 37. LSTM (100K) result in graphical form	55
Figure 38. GRU (100K) result in graphical form	55

Table of Contents

Acknowledgement	3
Abstract	4
Terminology	5
Abbreviations	6
List of Tables	7
List of Figures	8
Chapter 1: Introduction	13
1.1 Background and Introduction	13
1.2 Motivation and Purpose	14
1.3 Aim	15
1.4 Objective	15
1.5 Scope	15
1.6 Limitations	15
Chapter 2: Literature Review	17
2.1 Recurrent Neural Network (RNN)	18
2.2 Long Short Term Memory (LSTM)	18
2.3 Gated Recurrent Unit (GRU)	19
Chapter 3 : Methodology	21
3.1 General Methodology	21
3.2 Project Methodology	22
3.3 Technology Used	23
3.4 System requirements	24
3.4.1 <i>Hardware requirements</i>	24
3.4.2 <i>Software requirements</i>	25
	10

Chapter 4: Dataset Preparation	26
4.1 Introduction	26
4.2 Dataset Status	26
4.3 Data Preprocessing	27
4.4 Tokenization	29
4.5 N-gram sequence	29
4.6 Padding	30
Chapter 5: Deep Learning Models	32
5.1 BERT	32
5.2 LSTM	34
5.3 Bi-LSTM	36
5.4 GRU	37
Chapter 6: Model Training	39
6.1 Evaluation Matrix	39
6.2 Hyperparameters	39
6.3 Fine Tuning of Hyperparameters	39
Chapter 7: Model Integration	43
7.1 Overview of Model Integration	43
7.2 Loading the Model and Tokenizer	43
7.3 Backend Implementation	44
7.3.1 <i>Predicting the Next Word</i>	44
7.3.2 <i>Handling User Requests</i>	44
7.4 Frontend Integration	44
7.4.1 <i>Capturing User Input</i>	44
7.4.2 <i>Displaying Predictions</i>	44
7.4.3 <i>User Experience Enhancements</i>	45
7.5 Challenges and Solutions	45
Chapter 8: Web Development Application	46
	11

8.1 Technology Stack	46
8.2 Web Application Structure	46
8.3 User Interface Design	48
8.4 Backend Functionality	51
8.5 Implementation Details	51
8.6 Challenges and Solutions	52
Chapter 9: Result analysis and discussion	53
9.1. Model Comparison for 20K dataset.	53
9.2 Graphical Representation of Model performance for 20K dataset.	53
9.3 Model Comparison for 100K dataset.	55
9.4 Graphical Representation of Model performance for 100K dataset.	56
Conclusion	57
References	58

Chapter 1: Introduction

1.1 Background and Introduction

The national language of Bhutan is Dzongkha, and it not only serves as a means of communication, but it is also a crucial part of Bhutan's identity and cultural heritage. All the important Bhutanese literature, songs, religious scriptures and Dzongkha newsletters are documented and disseminated in Dzongkha, and it is through Dzongkha that Bhutanese people are able to connect with their cultural and religious heritage.

Among the various fields of natural language processing, next-word prediction is a fascinating area that has seen many remarkable developments in general and, indeed, most recently, in works on RNN-based language models. Next word prediction can be considered an artificial intelligence task with, in essence, a language modeling problem, in which one has to predict the most probable word or sequence of a word following a given text input based on the features of a given word. The current state-of-the-art of doing this in NLP today taking the sequence into context is attained through deep learning models, mainly recurrent neural networks (RNNs) and transformers. These models learn the dependencies between words within a corpus of text and, based on that, make word predictions to complete the next word.

Preferred models in deep learning for next word prediction include: -

1. BERT: BERT stands for Bidirectional Encoder Representations from Transformers. It is an improvement compared to unidirectional models, which only learn how to relate the meaning of a word from the previous work.
2. GPT: it stands for Generative Pre-trained Transformer and tasks such as text generation, language translation and to answer questions can make use of this model. Next word prediction can also be done using this model and has proved to be more accurate than BERT.
3. XLNet: it stands for eXtreme Multi-Lingual Language Model. This model can use both preceding and succeeding words to learn the context of the word and is a generalized auto-regressive language model. It is also found to be more accurate than BERT and GPT while doing some of the tasks.

The n-grams models are yet another alternative besides the deep learning models for implementing the task for the next word prediction. They are based on the theory that the probability of the occurrence of a word depends on the previous n-1 words. For instance, a 2-gram model will predict the next word based on two last words. The only downside with n-gram models is that, though simple and easy to implement, they lag in the accuracy that the deep learning models present.

Developing the next word prediction system is an ever-active research area in the contemporary state of the art. Deep learning models are growing in potency and are being harnessed for a range of NLP tasks, including the prediction of the next word in a sentence.

In this chapter, a brief introduction will be made to our proposed research project. This focuses on developing a Dzongkha Next Word Prediction System for the Dzongkha language. It outlines the motivation for developing such a system and explains why a next-word prediction system is needed during typing in the Dzongkha language. Moreover, some AI and ML models will also be elaborated in this chapter. As Dzongkha is a complex language and due the absence of Next word Prediction System for Dzongkha, challenges are faced while writing and typing Dzongkha letters. The aim and objectives of the research project will also be introduced along with the scope of the project. This section will conclude with some of the perceived possible benefits of such a system in the promotion and preservation of the national language of Bhutan.

1.2 Motivation and Purpose

Typing in Dzongkha language is laborious and error prone. To address this, next word prediction systems offer a potential remedy. By suggesting Dzongkha words and reducing manual effort, these systems enhance typing speed and accuracy. This aids in preserving language authenticity. Implementing such technology can alleviate typing challenges, maintain language richness, and promote meaningful expression in Dzongkha.

Even though Dzongkha is being promoted through writings, messages and documentations, it is evident that Dzongkha typing is difficult (Wangchuk et Al., 2021). Unlike typing alphabetic scripts, in Dzongkha each syllable is composed of

multiple characters and because of that one would require to use more key presses to type the words which can be quite hectic and time consuming. Not to mention Dzongkha spelling is unnecessarily cumbersome (The Druk Journal,2023).

Considering all the challenges that are faced in Dzongkha typing there is a need for a system that not only makes Dzongkha easy to type but also makes it a preferable option for the users. For these reasons, this project proposes a Dzongkha Next Word Prediction System that is able to predict the next syllable, reduce keystrokes and enhance the experience of Dzongkha typing.

1.3 Aim

To develop a Dzongkha Next Word Prediction System using Machine Learning.

1.4 Objective

Objectives of this projects are to:

- To acquire a dataset from the Dzongkha Development Commission.
- To identify and train the suitable model on the acquired dataset.
- To design and develop a graphical user interface.

1.5 Scope

- Next Word Prediction model: A system will be developed whereby it will be able to perform the next word prediction task by predicting and suggesting a list of probable words based on the feature or context of the preceding words.
- Design a Graphical User Interface for the system: A graphical user interface will be developed which will be convenient and user friendly and it will then be integrated with the model that we have trained for next word prediction.

1.6 Limitations

Performing NLP tasks on Dzongkha Language has several challenges and limitations, including:

- A, Lack of resources: Due to limited training data availability, Dzongkha, being a low- resource language, poses challenges for neural machine translation systems to effectively learn the intricacies of the language and generate precise translations.
- B. Morphological Complexity: Dzongkha is morphologically complex, with many inflectional and derivational forms. This may render it difficult for machine

translation systems to yield proper translations, especially for rare or low-frequency words.

C. Cultural differences: Dzongkha is one of the cultural words, where most of the words and sentences relate to cultural or contextual meanings, which sometimes are not in English. This makes it difficult for machine translation systems to reflect the intended meaning accurately.

D. Data error in training: The performance of the machine translation system is hugely dependent on how good the training data is. If the training data is wrong or error-prone, then the quality of translation will be low.

E. Domain-specific vocabulary: Dzongkha contains a domain-specific language for Buddhism, agriculture, and other aspects of the culture in Bhutan. The precision in the said areas is poor for the systems since they may not have been well-trained in the domain-specific terms and consequently make bad translations in these areas.

Chapter 2: Literature Review

According to a paper by Wangchuk et al., their next syllable prediction system aimed to address all the issues concerned with Dzongkha typing, including the use of several keystrokes. An LSTM neural network was incorporated to make the model learn. The model learns the syntactic and semantic features of language through the sequence generated from the Dzongkha sentences. The model had an accuracy of 78.33 percent, and deployment was done using the Django system (Wangchuk et al., 2021).

A Dzongkha next syllable prediction system is mainly aimed to reduce keystrokes and key presses to make Dzongkha typing fast and as convenient as possible. Segmented datasets were provided by the Dzongkha Development Commission of Bhutan, and these were processed through variants of RNN models. The Bi-LSTM model proved to be the most suitable model with two hidden layers and 512 hidden layer neurons, which contributed to an accuracy of 73.89 percent (Wangchuk et al., 2023).

In another study most evident architectures and regularization methods with extensive automatic black box hyperparameters tuning were evaluated. After the evaluation it was found that standard LSTM architectures had gone through proper regularization and it had the best performance from the nominated models (Melis et al., 2018).

A study conducted by Gannai & Khursheed, says that Recurrent Neural Networks (RNNs) represent the forefront of processing consecutive data, being used in technologies like *Apple's Siri* and *Google Voice Search* and LSTM. The feedback connections in it acts as a versatile tool, which can work with single data points like images and entire sequences such as speech or video. This flexibility is clear in tasks like recognition of connected handwriting and speech understanding. A fundamental aspect of NLP is that the N-gram Language Model predicts word groups one after another, while neural networks play a crucial role in predicting the next words in a language model. (Gannai & Khursheed, 2019)

The next word prediction for Hindi was done using two deep learning techniques: LSTM and Bi-LSTM (Sharma et al., 2019). These neural networks predict the next word of a given sequence of at least six words. Both models were implemented using the categorical cross-entropy loss and SoftMax activation functions. Both models were further trained with a learning rate of 0.001. The accuracy of the LSTM was 70.89%,

and the accuracy of the Bi-LSTM was 79.54%. The validation accuracy for the two models is, respectively, 59.46% and 81.07%. For the same dataset, Bi-LSTM model performs better and learns more quickly than LSTM.

2.1 Recurrent Neural Network (RNN)

Unlike the traditional neural network, the current step output is the following step input in a Recurrent Neural Network (RNN). This network becomes very useful for tasks such as the next-word prediction system, as the data of the current word is required for predicting the next word. It has a hidden layer that contains a hidden state that stores some information about the sequence.

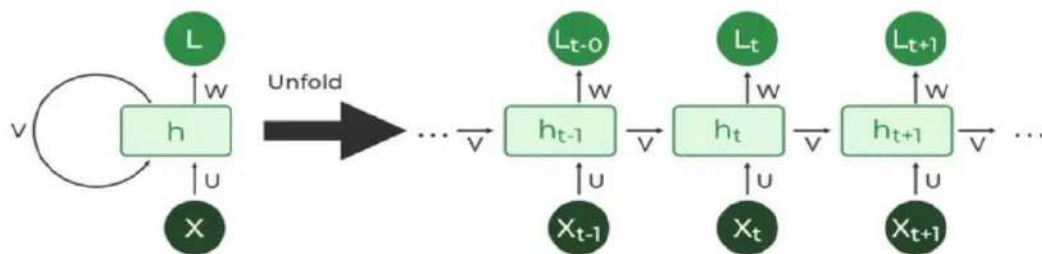


Figure 1. RNN Architecture

2.2 Long Short Term Memory (LSTM)

Another variant of RNN is the Long Short-Term Memory: a kind of RNN that arguably does a better job of handling and processing the sequential data type. The LSTM network structure is constituted by an array of LSTMs where each of these cells is endowed with several gates. These are the input gate, output gate, and forget gate. Information is regulated into and out of the cell by the input and output gates. The forgetting and retaining information from the past states helps in maintaining long-term dependencies in the sequential data. The LSTM memory cell retains information from the present instance in time to affect its output in the subsequent cases in time.

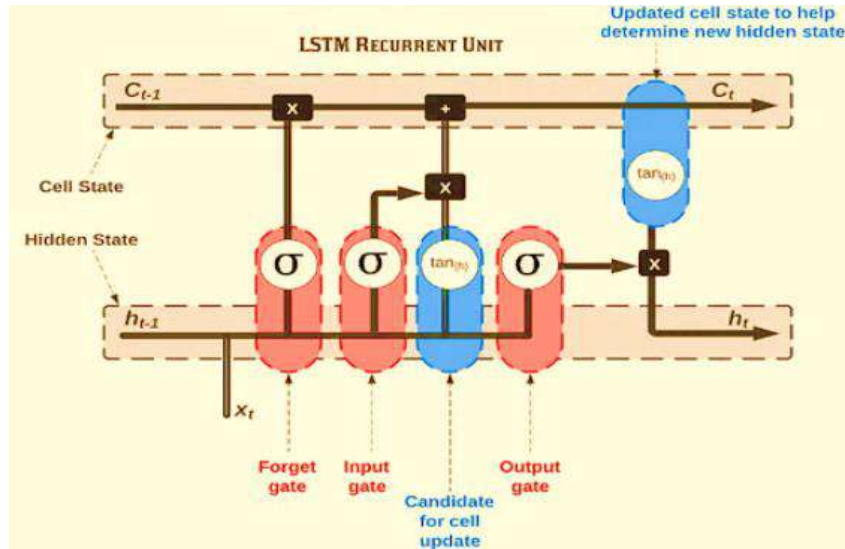


Figure 2. LSTM model Architecture

2.3 Gated Recurrent Unit (GRU)

Gated Recurrent Unit (GRU) is a type of RNN and a simpler alternative to LSTM. It has to be equipped to process sequential data, just like LSTM is. This updates the network's hidden state at each time step by selectively gating it and the mechanism that flows inside and outside of the network. The two gating mechanisms in the GRU network are the reset gate and the update gate. The reset gate determines the value of the earlier hidden state that has to be forgotten, while the value of the new input, which updates the secret state, is driven by the update gate. Updating the secret state takes place to compute the output of a GRU network.

Such equations include those to get the reset gate, update gate, and hidden state, as follows:

$$\text{Reset gate: } r = \sigma(W_r \times [h_{t-1}, x])$$

$$\text{Update gate: } z = \sigma(W_z \times [h_{t-1}, x])$$

$$\text{Candidate hidden state: } h_t' = \tanh(W_h \times [r_t \times h_{t-1}, x])$$

$$\text{Hidden state: } h_t = (1 - z) \times h_{t-1} + z \times h_t'$$

where all W_r , W_z and W_h are learnable weight matrices, x is an input at time-step

t , h_{t-1} is previous hidden state, h_t is current hidden state.

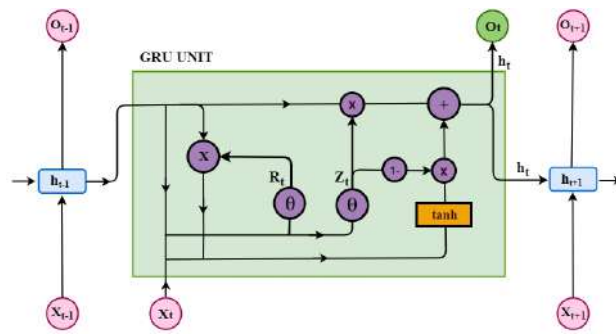


Figure 3. GRU model architecture.

Chapter 3: Methodology

This chapter goes on to provide a detailed explanation of the methodology followed for developing the Dzongkha Next Word Prediction System.

3.1 General Methodology

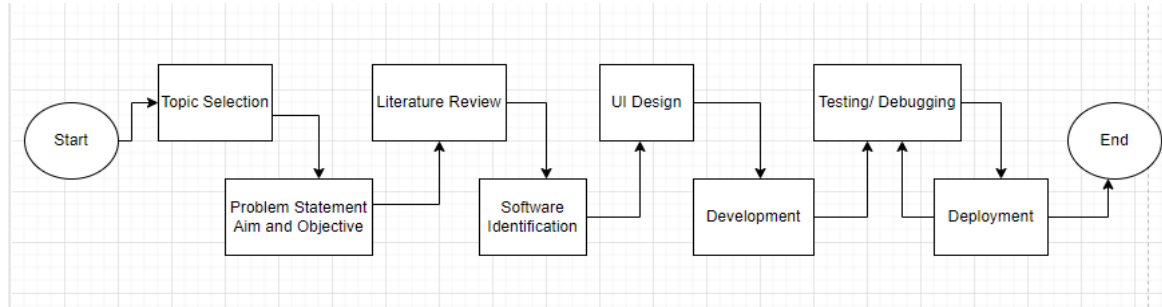


Figure 4: General Methodology

Generally , for the projects, we first select the topic that we are interested in. We define the problems that can be addressed or solved by the project, aim and objectives of the project. We also do a literature review to get in depth knowledge and insights about the topic. For generating the end product ,we first need to identify the tools that we are familiar and comfortable with so we do not have to face any complications while carrying out the project. For the software projects, this mainly involves selection of appropriate software tools and frameworks for development. We also design a user interface so as to make it convenient for the users to interact with our system. After having designed, we develop the system , this involves writing the codes for the system. To ensure the quality and performance of our system , we carry out the testing phase and check for errors to rectify them. After testing, we deploy the system where our system will be reached out to the wider range of users, they will also get to know the usage of the system through the seminars that we host.

3.2 Project Methodology

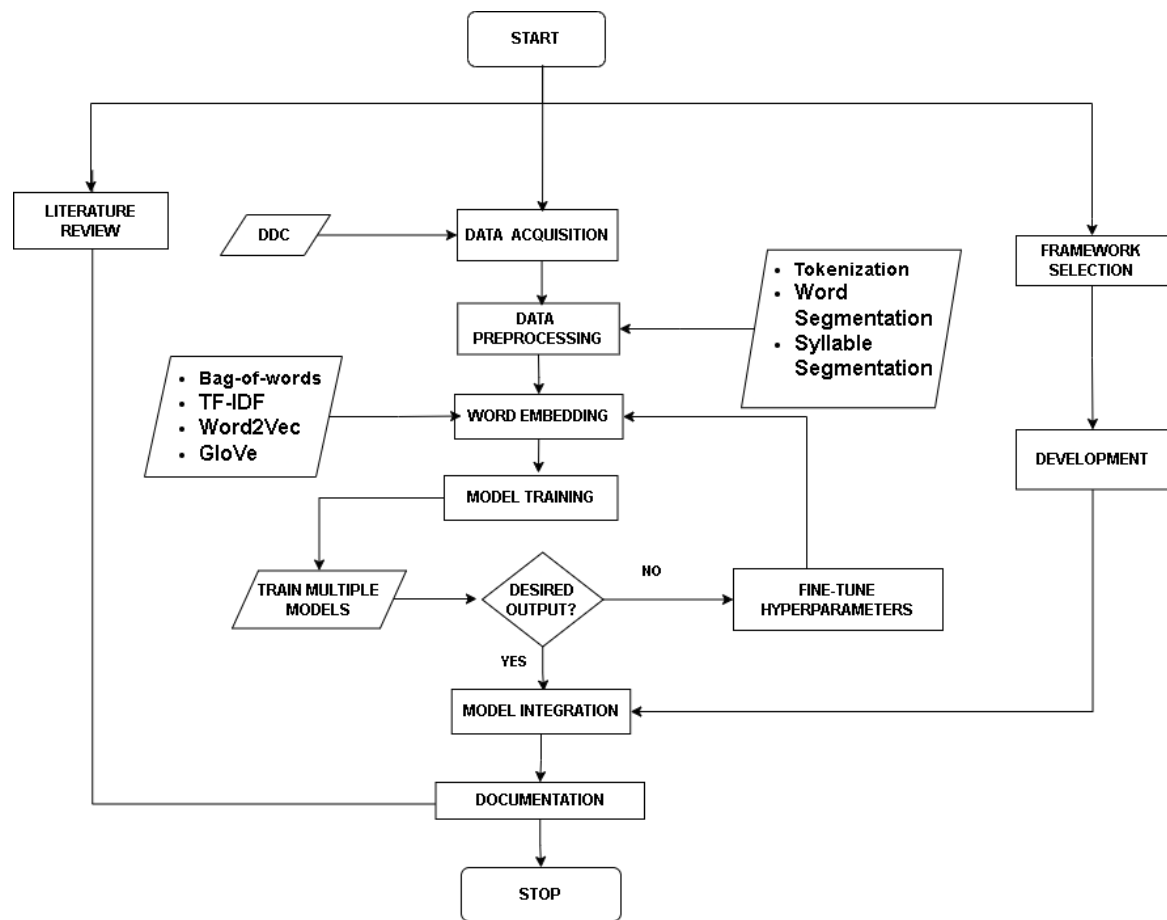


Figure 5: Project Methodology

1. Literature Review: various authentic and relevant papers will be reviewed, and the concepts learnt will be applied throughout the span of the project.
2. Data Acquisition: The first step is to collect a corpus of text in Dzongkha, which will be acquired from the DCDD (Department of Culture and Dzongkha Development).
3. Data Preprocessing: Specific techniques used in data preprocessing, such as tokenization and normalization, will mainly be done, meaning that it can help to improve the precision and performance of models.
4. Word Embedding: Word embedding is a technique where words are represented as vectors of real numbers. These vectors are typically learned from a corpus of text, and they can be used to represent the meaning of words, their relationships to other words, and their context in a sentence. Some of the word embedding

techniques that are usually implemented are Bag-of-words, TF-IDF, Word2Vec and Glove.

5. **Model Training:** Two or more models will be selected and trained on the same dataset. Then the model that gives us the highest accuracy or optimal result will be selected. If the model doesn't give us the desired output, then fine tuning of the hyperparameters will be done. The results of the fine-tuned parameters will then be sent for word embedding. The process will continue until the desired output is obtained.
6. **Framework selection:** An appropriate framework will be selected for the development of the system. The selected framework will aid in the easy integration of the model with the system in the later phase.
7. **GUI Design:** A user-friendly interface(prototype) will be designed for the system which will be used as a reference during the front-end development stage.
8. **Development:** The system components are coded/implemented according to the design of the GUI using the selected framework.
9. **Model Integration:** The development of the system and model training completed, the model will be installed in the system.
10. **Documentation:** In this phase the user manuals, technical documentation and training materials are created.
11. **Deployment:** The system is deployed and made available to the users.

3.3 Technology Used

We used Figma to design a prototype for our web app,.HTML, CSS and JavaScript for the front-end development. BOOTSTRAP was used to make our web app responsive. Django, a python-based web framework was used for the backend development and SQLite is used as the database. Model training was done using google colab.



Figure 6. HTML logo



Figure 7. CSS logo



Figure 8. google colab logo



Figure 9. Javascript Logo



Figure 10. Bootstrap logo



Figure 11. Django Logo



Figure 12. SQLite logo

3.4 System requirements

Outlined below are the system requirements for the implementation of Dzongkha to English Translation system, encompassing both hardware and software aspects. The project necessitates specific prerequisites to ensure successful execution. The detailed requirements for undertaking such a project are as follows.

3.4.1 Hardware requirements

1. 8GB ram minimum, 16GB or higher recommended.

2. Intel Core i5 or i7.
3. 500 GB storage.
4. GPU NVidia RTX 3090 (48 GB VRAM)

3.4.2 Software requirements

1. Deep Learning library - Keras, Tensorflow, Pytorch, Huggingface
2. Python 3, JavaScript, HTML, CSS
3. Google Colab
4. Figma
5. Django Framework
6. Dataset Storage - Microsoft excel, csv.

Chapter 4: Dataset Preparation

4.1 Introduction

In this chapter, we discuss the data preparation process that we undertook for developing the Dzongkha Next Word Prediction System. This process includes data collection, data preprocessing and dataset statistics. The explanation of each of these steps and the techniques used to prepare the data for our system will be done in detail. It is essential to prepare the dataset to carry out any of the ML projects, including Next Word Prediction systems. The dataset is the most asset in any machine learning or deep learning project. The capacity and consensus of the model depends on the size of the data and the quality of the data, and these determine whether the system works on new samples accurately or not. The given dataset for this project includes more than one hundred thousand various sentences provided by the DCDD. Preprocessing has three steps to form the required dataset: data pre-processing, tokenization, and padding.

4.2 Dataset Status

At the time of training the model to start with there were many constraints regarding the size of data we could take because we did not have a computer system which could hold a large data set.

SL. No	Variable	Value
1	Total sentence	20,000
2	Total words	559,334
3	Total no. of unique words	7816
4	No. of sequence	115,351

Table 2: 20,000 Dataset

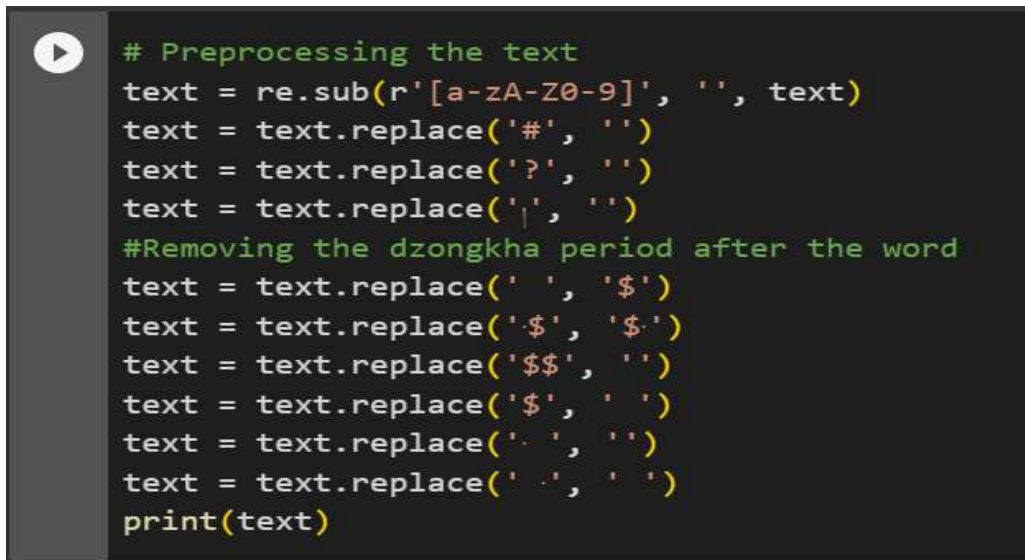
We then increased the size of the dataset and utilized the GPU system for training our model.

SL. No	Variable	Value
1	Total sentence	100,000
2	Total words	1,331,282
3	Total no. of unique words	28,344
4	No. of sequence	623,820

Table 3: 100,000 Dataset

4.3 Data Preprocessing

The sample code in the figure below shows how the data preprocessing was done.



```

# Preprocessing the text
text = re.sub(r'[a-zA-Z0-9]', '', text)
text = text.replace('#', '')
text = text.replace('?', '')
text = text.replace('!', '')
#Removing the dzongkha period after the word
text = text.replace(' ', '$')
text = text.replace('$', '$ ')
text = text.replace('$$', '$ ')
text = text.replace('$', '$ ')
text = text.replace('.', ' ')
text = text.replace(' ', ' ')
print(text)

```

Figure 13. Data Preprocessing

The execution of the code snippet shows in the figure above mainly achieved tasks such as:

- a) Removal of alphanumeric characters.

- b) Removal of ‘#’ and ‘?’.
- c) Removal of ‘|’ characters.
- d) Removal of (:) at the word level.

The figure below shows how the dataset looked before preprocessing.

```

0          ང རྫོང་ ས རྩ ལ མིན །
1      ཕྱི་མི འདི # ཡིད་ཁར་འོང་རྟོག་རྟོ འདུག །
2          ཕྱི་མི འདི མ ཤི བས །
3          ཕྱི་མི འདི ཉལ རུག །
4          རྒྱ་མི འདི ཆག རུག །
        ...
19995    རྫོང་ སྤུང་ # ཐག རིང་ སར་ མིན་མ སམ་ # ང་བཅས་...
19996    རྫོང་ འདི # བེ་ཡར་ འབྲུང་ བར་ # རྒྱང་གྲུ ར ་...
19997    རྫོང་ སྤུང་ # གཅི ཅིག རྒྱལ་ རི་ # འབད་ དེས །
19998    རྫོང་ འདི # ཐོག་ཁ ལ་ # རོ་ལེ་ ཉལ་ཁང་ རྒྱང་...
19999    རྫོང་ འདི # གསལ་བཤད་ བཅོ་ རིམ་ རྒྱང་ # ཕྱང་...
  
```

Figure 14. Data before preprocessing

The figure below shows how the dataset looked after preprocessing.

```

0          ང རྫོང་ ས རྩ ལ མིན
1      ཕྱི་མི འདི ཡིད་ཁར་འོང་རྟོག་རྟོ འདུག
2          ཕྱི་མི འདི མ ཤི བས
3          ཕྱི་མི འདི ཉལ རུག
4          རྒྱ་མི འདི ཆག རུག
5      རྒྱུ་ཕྱིར་ ཕྱི རྒྱང་ རི འདི མཆོམས་འཛོག་ འབད་ རུག
6          གདམས་ འདི རྒྱང་ ལ་ ཡོད
7          ང ལྷ འབད་ ས རྩ ལ མིན
8      དེ་འབད་མ་ལས་ ང སྤུང་ འབྲུང་ བཅོང་ ཡི
9          དེ་འབད་མ་ལས་ ང སྤུང་ འབད་ ཡི
10         དེ་འབད་མ་ལས་ ང མི རོམ་ བས
11         དེ་འབད་མ་ལས་ ང རྩ རྫོང་ ཡི
12         དེ་འབད་མ་ལས་ ང འབྲིས་ རྫོ ཡི
  
```

Figure 15. Data after preprocessing

We remove alphanumeric characters, special characters, '|', '?'. Tsha and shay are removed because we want to make the Dzongkha words equivalent to English words. We remove alphanumeric characters, special characters because in a word, it doesn't contain special characters and symbols. Non-alphabetic and numeric characters may not provide meaningful information for the language model and can introduce unnecessary complexity.

4.4 Tokenization

Tokenization is a pre-processing task that has the goal of segmenting the text into smaller units. We employ a successive tokenization method that comes from TensorFlow to divide every of the Dzongkha words depending on the vocabulary list of every unique word accompanied with its unique integer values.

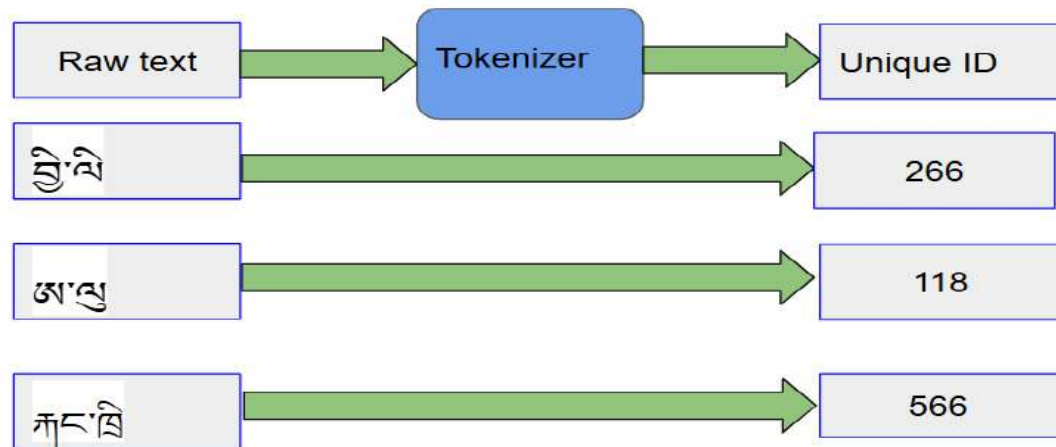


Figure 16. Tokenization Process

The figure above depicts the raw text that is tokenized with a unique ID. For example, we have 'ཐུ་ལི' which is tokenized with a unique ID of 266. And this is how we have implemented it in our code.

```
tokenizer.fit_on_texts(text_data)
total_words = len(tokenizer.word_index) + 1

print("Total number of unique words: ", total_words)
print("Word: ID")
print("=: ", tokenizer.word_index['='])
print("ཐུ་ལི: ", tokenizer.word_index['ཐུ་ལི'])
print("ཨ་ལུ: ", tokenizer.word_index['ཨ་ལུ'])
print("ཀྲང་ཁྱི: ", tokenizer.word_index['ཀྲང་ཁྱི'])
```

Figure 17. Code implementation for tokenization

4.5 N-gram sequence

We used the N-gram sequence for sequence generation. The N-gram sequence is created to curate the dataset into features(X) and labels(Y). As we can see the last token in each sequence is Y(label) and all the other tokens before Y are X or the features.

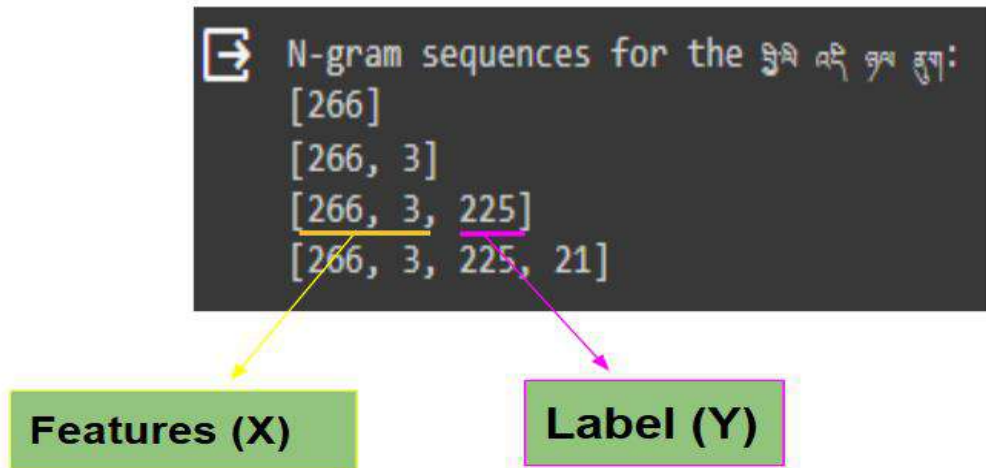


Figure 18. N-gram sequence generation

The model is trained to predict Y based on X. where y is label and X is the features.

4.6 Padding

In the DzoSyll dataset, the string lengths range between 1 and 171 letters or characters for each term in the dataset. We decided not to expand the maximum sequence length to the length of longer sentences, while most of the sentences in our dataset train and dev set had a maximum sequence length of 22. To be able to construct matrices with equal dimensions for all input sequences, the n-gram sequences are completed by zeros in the front. Padding is carried out to make sure that the feature sets of all the inputs are equal in size and neural networks demand inputs of equal dimensionality. As a result of tokenization, it is necessary to append additional tokens for each sequence to reach the length of the longest sequence in the corpus. Adding a beginning special token also allows the model to ignore all the other unnecessary data, also preserves the time order and helps the model to understand the relationships between words and context.

Chapter 5: Deep Learning Models

In this chapter, we build on what we described in previous chapters regarding deep learning models for the benefit of the reader offering an understanding of some of the newest findings that have recently emerged to revolutionize AI. We will also mention the system architecture of the models such as LSTM, Bi-LSTMs, BERT and GRUs. Further, we will also explain the fundamental ideas and how to train as well as tune these models for getting improved outcomes.

5.1 BERT

In the case of the architecture of BERT, a multi-layer bidirectional transformer encoder like the transformer model is used. To be more precise, a transformer comprises an encoder-decoder network as well as incorporating self-attention for the encoder and attention for the decoder. BERT has two main variants: This paper focuses on two models namely BERTBASE and BERTLARGE. There are two major versions of the BERT architecture that have been released, namely BERTBASE which has 12 layers in its encoder stack and the BERTLARGE which has 24 layers in the same stack. These configurations include more layers than the Transformer model which, as previously mentioned, contained only 6 encoder layers in accordance with the authors of the publication. In addition to having more layers as compared to other transformers, BERT has a larger feedforward network and more attention heads than the first structure in transformers. Specifically, BERTBASE is configured with 768 hidden states and 12 attention heads while BERTLARGE comes with 1024 hidden states and 16 attention heads. Meanwhile, the initial Transformer architecture had a hidden dimension of 512 sizes and eight attention heads.

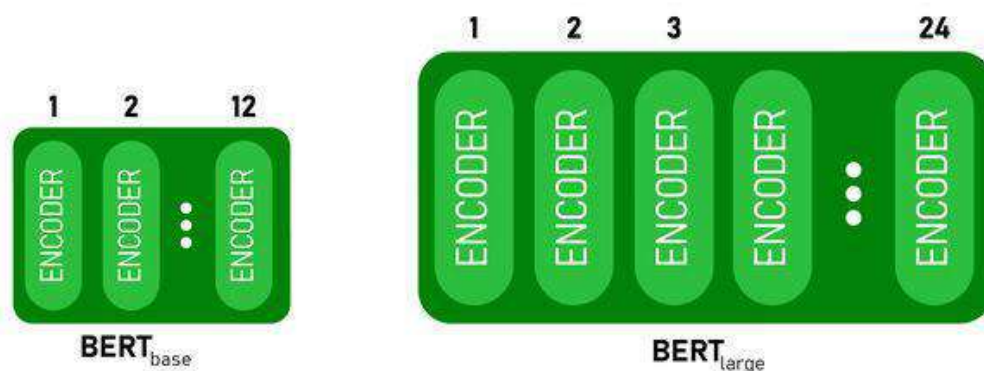


Figure 21. BERT BASE and BERT LARGE architecture

BERTBASE has almost 110 million parameters, and BERTLARGE has about 340 million parameters. The BERT model input begins with the CLS (classification) token at the start and is followed by a sequence. The CLS token is essential for tasks that require classification. This can be done after processing an input sequence through as many layers as our model contains, each applying the self-attention mechanism, transforming output, feeding it to the feedforward network and passing it further to the next encoder layer. At the final stage the model produces an output with a hidden size, which is 768 in BERTBASE. For classification problems typically the output associated with the CLS token type is employed.

Going by the analysis made on the BERT model research data that has been gathered and analyzed from the internet. In 2022, BERT and similar models that rely on the approach of MASK, where the actual written word is omitted and replaced with '[MASK]' did not prove to be all that accurate in predicting the next word.

As per Noe, one issue associated with employing BERT for text generation is that it was not designed for predicting the next word.

(Noe, 2021) said You can also take a random word generator and use it to "generate text". Does it generate text? Well, yes, but no. The same applies to BERT. You can use BERT to generate text by placing a [MASK] token at the end of a sequence of tokens and using its prediction to autoregressive build more text, but don't expect a good output.

Example:

Input Text:

"BERT is not meant for next word prediction."

Tokenized Text:

["[CLS]", "BERT", "is", "not", "meant", "for", "next", "word", "prediction", ".", "[SEP]"]

BERT Output Tokens:

["[CLS]", "[MASK]", "[MASK]", "[MASK]", "[MASK]", "[MASK]", "[MASK]",

"[MASK]", "[MASK]", ".", "[SEP]"

5.2 LSTM

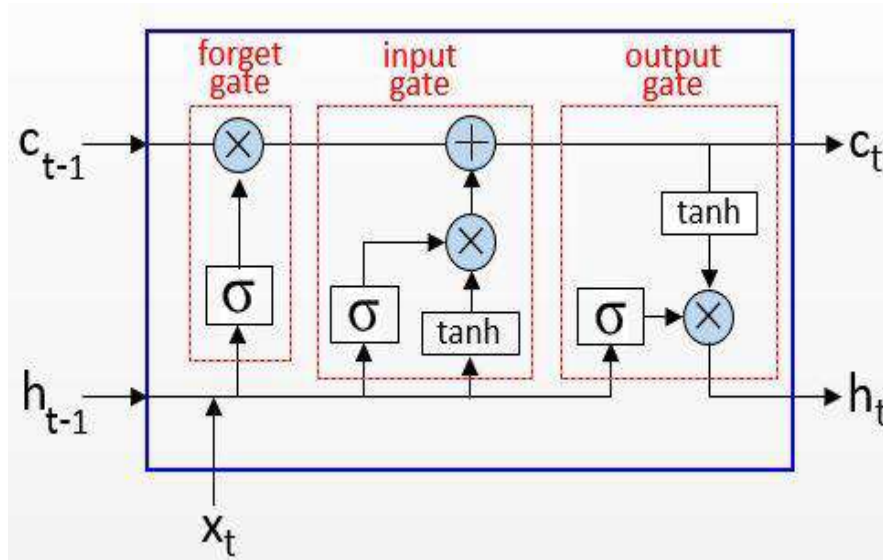


Figure 22. LSTM cell

This is a representation of the LSTM cell and the different layers of the architecture. LSTM stands for long-short term memory and can be considered as a specific form of Recurrent Neural Network.

Next Word Prediction is carried out using LSTM cells that are capable of learning dependencies in a series of inputs, which is indispensable for the given task.

The LSTM cell in the image has three main components:

1. The forget gate: The forget gate determines the amount of memory coming from the previous N cell that should be 'forgotten'.
2. The input gate: This formula presents the impression that the input gate decides the extent of new information that flows into the cell state.
3. The output gate: Output gate regulates the amount of cell state which should be passed to the next layer of the RNN.

The LSTM cell performs the following three functions in its operation; The first function calculates the forget gate; The second function calculates the input gate; The

third function calculates the output gate. These gates are computed through the help of sigmoid activation function which gives it a value in between 0 and 1.

The forget gate controls the extent to which the previous cell state should be retained in the memory.

If the forget gate output is near to 0, then a greater part of the previous cell state will be discarded. If the forget gate output is closer to 1, then there will be retained part of the information of the previous cell.

The input gate controls the amount of new data that is to be allowed into the cell state. If the input gate output is nearly equal to zero, then nothing much will be contributed to the cell state. If the value of the input gate output is almost 1, then all new information will pass directly through to the cell state.

The output gate controls how much of the cell state is passed onto the next layer in the network. If the output gate 'output' is very close to zero, this means no information will be output from the cell state. On the other hand if the output gate output approaches one then all the information contained in the cell state will be output.

$$\text{new_cell_state} = \text{forget_gate} \text{ previous_cell_state} + \text{input_gate} \text{ new_information}$$

$$\text{output} = \text{output_gate} * \text{new_cell_state}$$

The LSTM cell is a very effective component for tracking one step sequences. The LSTM cell can discover long term dependencies within the sequential data stream and make accurate predictions by means of the forget gate, input gate and output gate.

5.3 Bi-LSTM

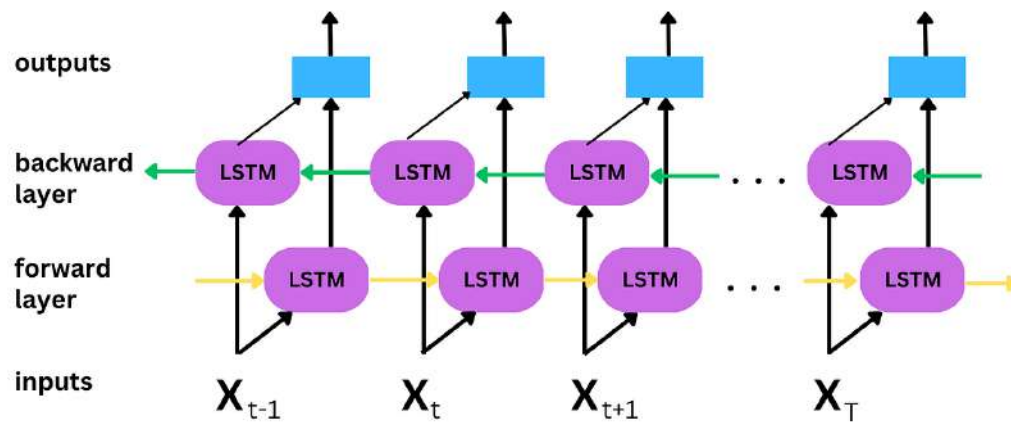


Figure 23. Bi-Lstm model Architecture

The image shows the architecture of a bidirectional LSTM (BiLSTM).

Specifically, there are LSTM models called BiLSTMs, which can work on matters through two directions simultaneously – the forward direction, or from left to right, and the backward direction or from right to left.

This makes it possible to get BiLSTMs to easily identify long-term dependencies in the sequential data than unidirectional ones.

The two LSTMs form a BiLSTM in which one LSTM works at forward and the other works at backward processing.

The BiLSTM, the input is made up of vectors representing the sequence from any of the analysis time steps.

The BiLSTM operates as follows in the sense that the first layer LSTM is a forward LSTM which processes the sequence in the left to right direction.

The result generated from the learning of the first LSTM layer is then passed through the second LSTM layer which in turn processes the sequence in right-to-left fashion.

This output is then used as the output of the BiLSTM by feeding it to the third LSTM layer with reversed input; this layer is also an LSTM layer.

5.4 GRU

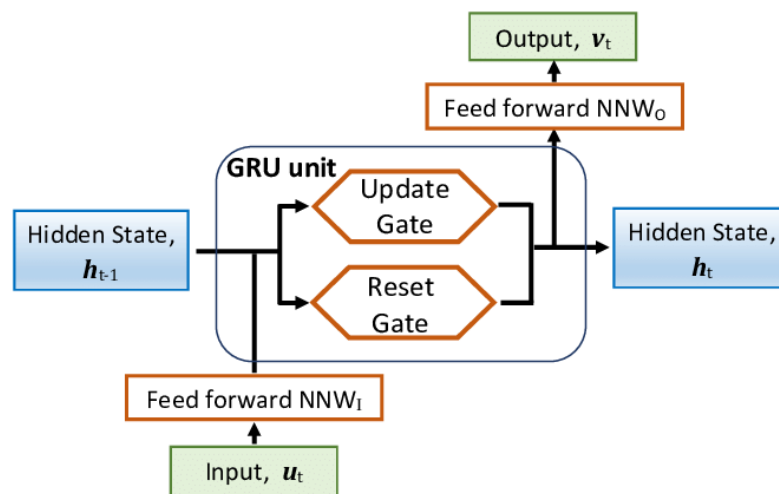


Figure 24. GRU architecture

Here is getting the Architecture of GRU, this model has two gates namely,

Below shows a block diagram of the gated recurrent unit which is also referred to as GRU cell. GRU cells are another type of RNN cells that work similarly to LSTM cells but contain fewer gates to attend, and so are less complex.

The GRU cell in the image has two main components:

1. The reset gate: It must be noted that the reset gate decides to what extent the previous hidden state is inherited.
2. The update gate: The update gate regulates the amount of new information incorporated into the hidden state from a specific time step.

First, one would need to compute the reset gate r_t and the update gate z_t . These gates are then computed using a sigmoid activation function and range from 0 to 1.

This makes them appropriate for such tasks because they are easy to train and are capable of learning sequence dependencies that extend in the long-term.

Computationally, LSTMs have been observed to be expensive as against GRUs because of the reduced number of gates and computations.

Chapter 6: Model Training

6.1 Evaluation Matrix

Accuracy is chosen primarily since test metrics that maximize accuracy were chosen to evaluate the models in terms of how accurate they make the predictions. The function of the system is mainly directed to this goal of being able to estimate well the words when given some other words that had been used previously.

6.2 Hyperparameters

- Epochs: This is the utterance of the feed forward and the number of cycles the entire data set must pass through the neural network.
- Batch size: It defines how many training samples the network should be passed across before updating the model.
- Validation split: This is done to hold out some of the data for the validation set. In training, the model will not try to train on this data but rather assess its performance on it. This aids in regularization and therefore aids in reducing cases of overfitting.

6.3 Fine Tuning of Hyperparameters

Training

Training of the model is crucial to achieve a well-performing language model. Here's a snapshot of the code relevant to the model training process:

```
def create_model(total_words, input_length):  
    model = Sequential()  
    model.add(Embedding(total_words, 300, input_length=max_sequence_len - 1))  
    model.add(GRU(512, return_sequences=True))  
    model.add(GRU(256, return_sequences=False))  
    model.add(Dense(max(100, total_words // 4), activation='relu'))  
    model.add(Dense(total_words, activation = 'softmax'))  
    model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])  
    return model
```

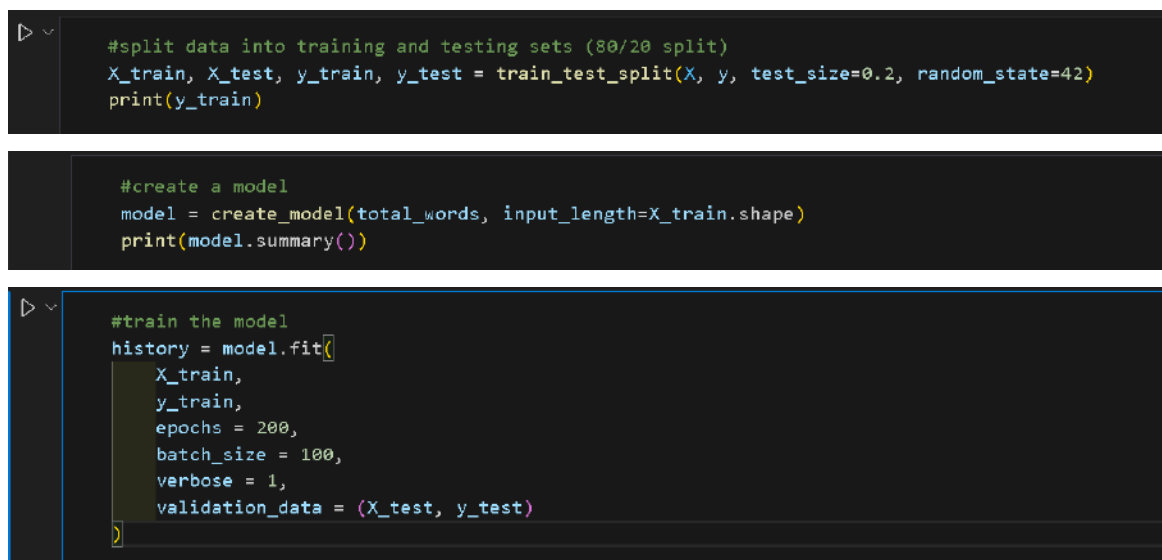
Figure 25. create_model code snapshot

Model Creation Function Explanation

The function `create_model` defines the architecture of a deep learning model for text prediction using the TensorFlow Keras framework. Here's a breakdown of each component within the function and its role in the model:

- **Sequential Model:** This forms a linear stack of layers, the hallmark of the crust. I chose the `Sequential` class because it is one of the easiest ways to create a single neural network, where the model has one input and one output, and hence, every layer has one input tensor and one output tensor.
- **Embedding Layer:** A layer in natural language processing models, this encompasses an input layer, a hidden layer, and an output layer. It transforms significant, diagonally stored vectors of integer representations of words with many zeros to small and dense vectors. The value set for the parameter `total_words` of the `Corpora` will determine the size of the given vocabulary, while 300 is the features or dimensions of the word embedding vectors. This is determined by the `input_length` parameter which is equal to `max_sequence_len - 1` in this case as the input length is the number of tokens of the input instances. The embedding layer helps the model to create compatibility between the relativity or semantic relationship of different words.
- **First GRU Layer:** This layer is a recurrent one with 512 units to deal with sequences through gated recurrent units (GRUs). It should be noted that GRUs are a type of RNN and are most used for sequence prediction tasks. The `return_sequences=True` parameter is used because the next layer is also recurrent, and the output is expected in sequence format.
- **Second GRU Layer:** This turns out to be another GRU layer, however, on this occasion with 256 units. This time, the setting of `return_sequences=False` will be used to have the layer spit out only the final output in the output sequence, which in effect turns the sequence into a vector. This is common especially in sequence prediction wherein one only requires the final state.
- **Dense Layer:** In this case, the number of neurons in this layer is 100 plus one fourth of the size of the vocabulary list, and this is a fully connected layer. Within the current model, the `ReLU`, which stands for Rectified Linear Unit, is used as an activation function to increase a model's capacity to learn more sophisticated relationships between features.

- **Output Layer:** The final layer is the output layer, which consists of a single layer of dense neurons equal to the vocabulary's number of words. A SoftMax transformation will retrieve the probability distribution of the next word from the word dictionary. The output of each neuron in the network is the probability of the transition to the next word, which corresponds to the neuron.
- **Compilation:** In this case, the model is trained using the loss function, the definition of which is ideally referred to as 'sparse categorical cross-entropy loss' for problems where each instance is expected to belong to one class exclusively. One typically uses the Adam method to optimize the solution in a stochastic space efficiently. In the process of training the model and during testing, a check is always performed and the measurement under consideration is called 'accuracy.'



```
#split data into training and testing sets (80/20 split)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(y_train)

#create a model
model = create_model(total_words, input_length=X_train.shape)
print(model.summary())

#train the model
history = model.fit(
    X_train,
    y_train,
    epochs = 200,
    batch_size = 100,
    verbose = 1,
    validation_data = (X_test, y_test)
)
```

Figure 26. Hyperparameters code snapshot

Fine-Tuning:

Fine-tuning simply means adjusting the model after training to make it more efficient or better suited to the specifics of the data. Here, it is explained in the following:

- We are making changes in the number of layers in GRU or layer size by altering the number of units.
- This had to be done to change the learning rate of the Adam optimizer as the training progressed.
- We may also further experiment with different architectures or add further regularizations such as Dropout.

Specific fine-tuning steps do not seem to be explicitly called by the script. Fine-tuning must be done by the layers and their configurations in the 'create_model' function.

Training and Validation Metrics Visualization

The script also contains code to plot the training and validation accuracy and loss which is really important as it represents the learning process and model's performance in epoch:

```
#Plotting the training and validation accuracy and loss

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()
plt.savefig('GRU_v13_training_validation_metrics.png')
```

Figure 27. plotting histogram code snapshot

Chapter 7: Model Integration

This chapter will highlight how the trained Dzongkha Next Word Prediction model will be incorporated into the content of the web application. Depending on the complexity of the application, it requires integration with other components, particularly the backend machine learning model to feed the user-interface for runtime forecast feedback.

7.1 Overview of Model Integration

The integration of the machine learning model into the web application involves several key steps:

1. Loading the Model: The trained model is loaded into the application.
2. Loading the Tokenizer: The tokenizer used during the training phase is also loaded to ensure consistency in text processing.
3. Backend Setup: Backend functions are embedded and used to implement the processing of the user's input, prediction implementation, and the transmission of results to the frontend.
4. Frontend Integration: The frontend is developed to communicate with the backend, capture user data, present the user with predictions, and handle user interactions.

7.2 Loading the Model and Tokenizer

The first step in the integration process is to load a trained model and tokenizer into the web application. It assures that a model can be run and deliver predictions in response to user input in real-time.

- Model Loading: The trained model is saved in a .h5 file and later loaded to the application.
- Tokenizer Loading: This loads the tokenizer from its .pkl file. Loading the tokenizer is very important, as it must be loaded correctly, ensuring that input text processing is correct; that is, words get translated to sequences of integers that the model understands.

7.3 Backend Implementation

Django is the backend of the application: it is highly effective and flexible in Python web frameworks. Starting from handling user data, the backend involves taking the trained model and making predictions, the result of which is presented to the frontend.

7.3.1 Predicting the Next Word

It includes the core functionality of processing the user's input, which is the seed text, with the tokenizer and generation of the following top N-word predictions using the trained model. In this way, it makes sure that the predicted words are appropriate and in context with the user's input.

7.3.2 Handling User Requests

Views are the way through which users give their input. The given input is processed, a prediction is made, and the result is sent back to the frontend from the backend. When the user types anything inside the input field, an AJAX call is made to the backend, which gives a response back to the frontend.

7.4 Frontend Integration

The application user interface is developed with HTML, CSS, and JavaScript, where responsive design ensures the framework is Bootstrap. This component is responsible for collecting the input from the user, passing it to the backend to get the prediction, and then showing the prediction to the user.

7.4.1 Capturing User Input

The front end is a text box where users can input Dzongkha text by typing. Every character key pressed by the user is recorded and then passed to the server through AJAX calls. This makes it possible to have live sessions and instant responses from the other party.

7.4.2 Displaying Predictions

Once the backend processes the input and generates predictions, the frontend dynamically updates to display these predictions. The predictions are shown as clickable buttons, allowing users to easily select and insert the predicted words into their text.

7.4.3 User Experience Enhancements

Through the frontend, an enhancement in the user interface will be created whereby users are able to effectively work in the application. Thus, options such as live feed, a flexible layout for typing Bhutanese language Dzongkha and with the next word options and easy-to-use controls make it convenient.

7.5 Challenges and Solutions

During the integration process, several challenges were encountered and addressed:

- **Size and Prediction:** Large data were used in the model making and more than one prediction had to be made in the shortest possible time, through optimization.
- **Ensuring Accuracy:** Other procedures that were useful to get various solutions specified included the fine tuning of the given model and the data preprocessing.
- **User Interface Design:** This led to much time being spent during the planning phase and consultations with users on how to arrive at the ergonomic and highly sensitive model.

Chapter 8: Web Development Application

To enable ease of typing in Dzongkha, the web application was created to offer auto completion based on the typist's previous input. This feature uses an advanced learning technique that sees the app develop its own model of analysis through use of a relatively big database for the language to produce a more accurate and thereby better user experience hence, increasing the usage of Dzongkha in the layout of digital communications.

8.1 Technology Stack

The development of the web application utilized a range of technologies to ensure a robust and scalable solution:

1. Frontend:

- HTML, CSS, and JavaScript for structuring and styling the web pages.
- Bootstrap for responsive design and pre-built components.
- Flowbite for additional UI components.

2. Backend:

- Django, a Python-based web framework, for handling server-side logic and integration with machine learning models.
- SQLite for database management.
- TensorFlow for model training and predictions.

8.2 Web Application Structure

The application consists of several key components, each responsible for specific functionalities:

1. Home Page:

- Displays a text input area where users can type Dzongkha text.
- Provides dynamic word predictions as the user types.
- Includes buttons for importing and exporting text and sharing content.

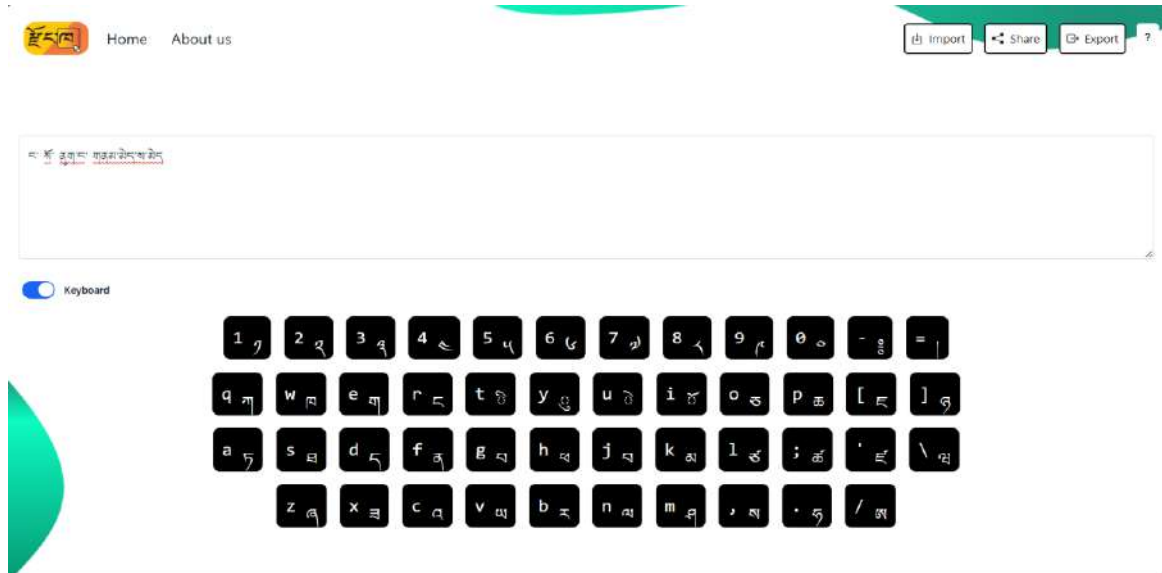


Figure 28. Landing page of Dzongkha Nextword Prediction

2. About Us Page:

- Contains information about the development team and the project guide.
- Showcases the motivation and purpose behind the project.

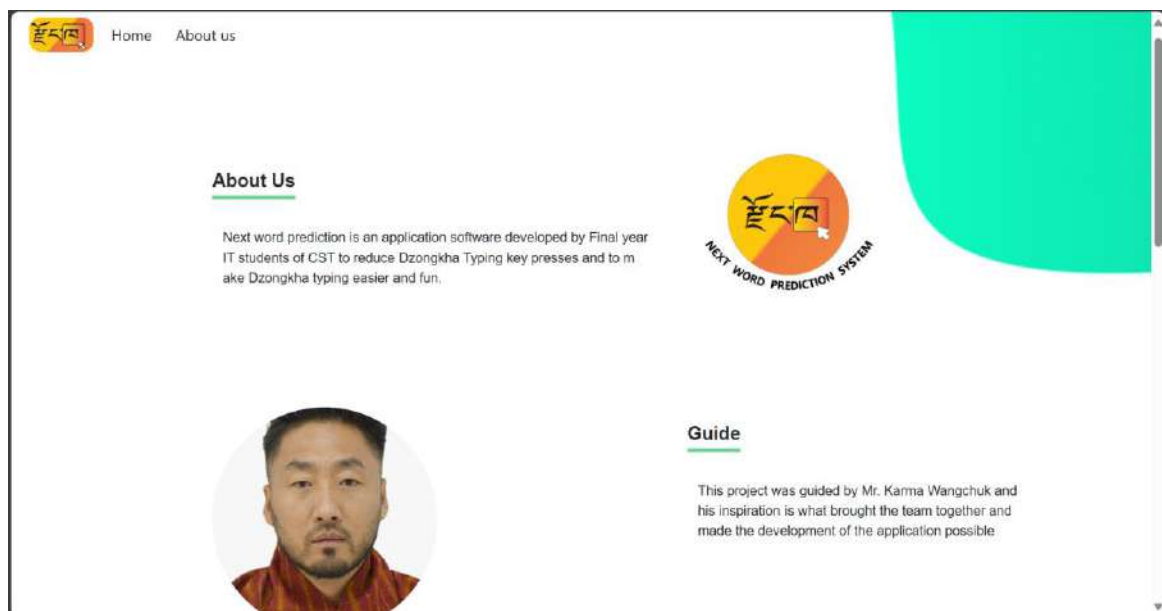


Figure 29. About Us page

8.3 User Interface Design

In view of this, the different elements that make up the ‘look and feel’ of the application, commonly referred to as the user interface (UI) was generally made as logical and as easy to understand as possible:

1. Text Input Area:

- A text area where users can input Dzongkha text.
- Real-time display of word predictions below the text area.
- Integration with a virtual keyboard for ease of typing Dzongkha characters.

2. Predictions Display:

- A dynamic container that shows the top word predictions based on the user's input.
- Buttons for each predicted word can be clicked to append the word to the text area.

3. Import/Export Functionality:

- Modal dialogs for importing text from DOCX or TXT files and exporting text to various formats (PDF, DOCX, TXT).

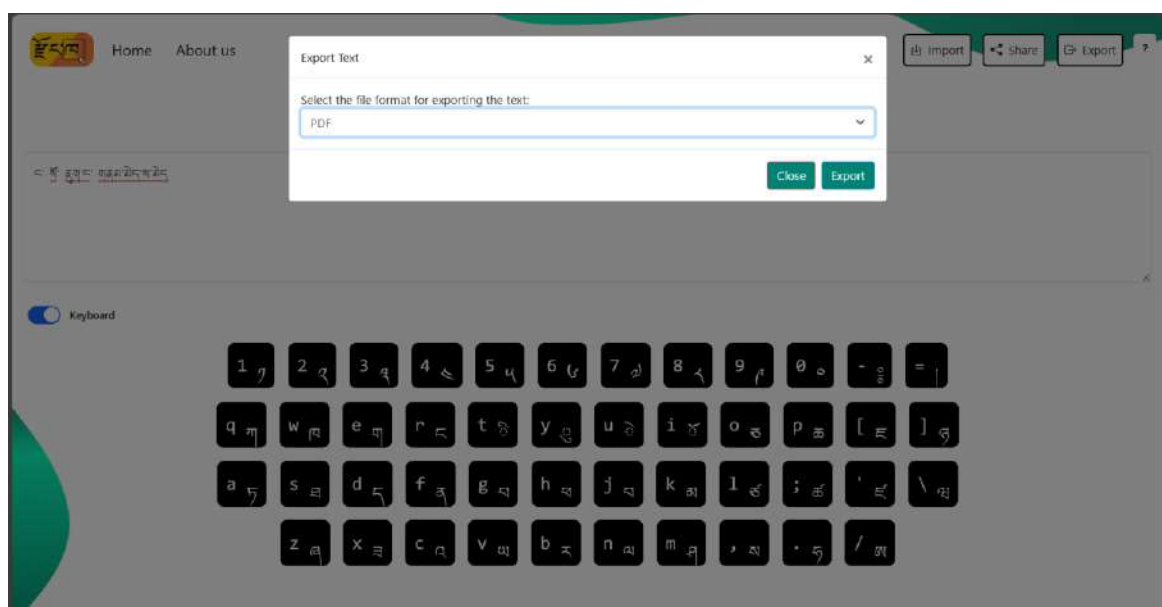


Figure 30. Exporting Dzongkha Text

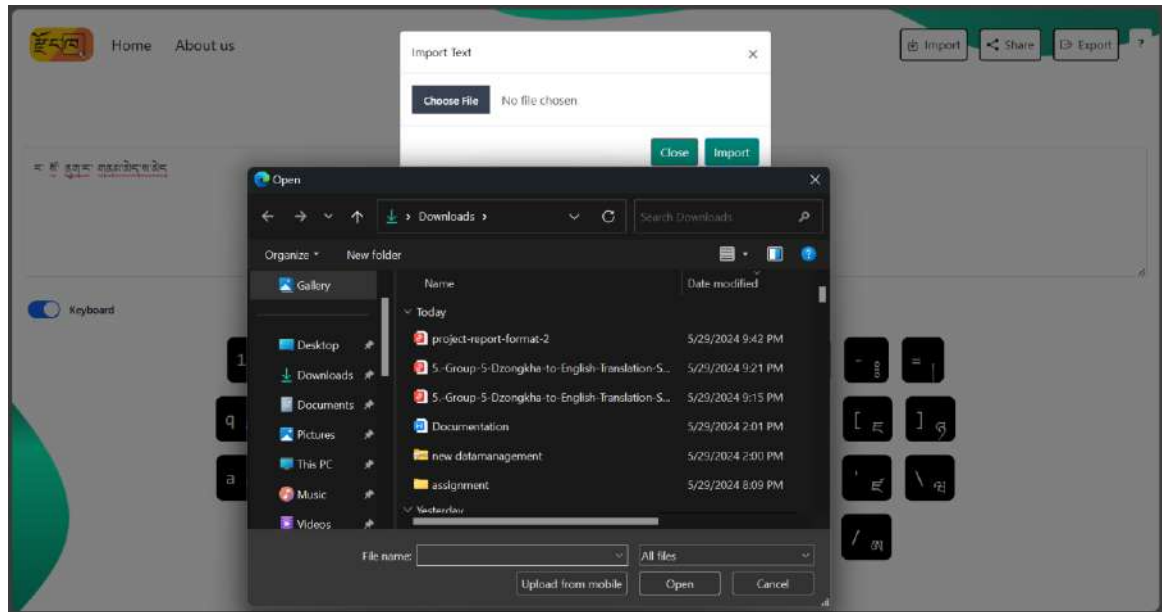


Figure 31. Importing Dzongkha Text

4. Share Function

- The share function enables easy sharing of typed text using the Web Share API:
 - **Button Placement:** Conveniently placed in the navigation bar.
 - **Functionality:** Retrieves content from the text area and uses the Web Share API to share it via supported applications.
 - **Fallback Handling:** Alerts users if the Web Share API is not supported by their browser.

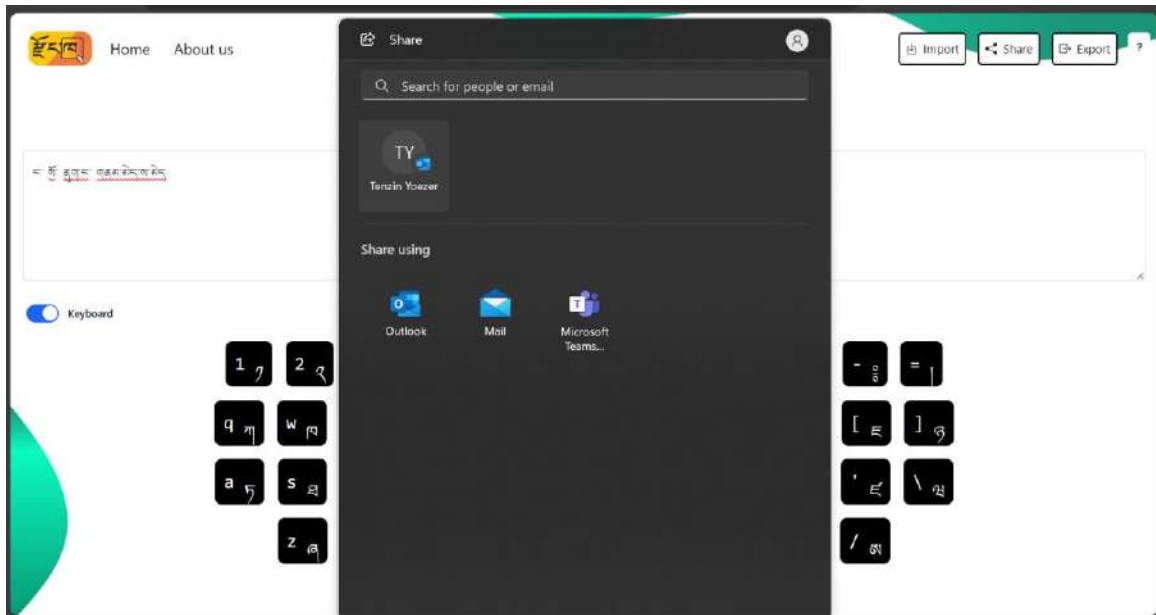


Figure 32. Share functionality

5. Tutorial Section

The tutorial section provides step-by-step instructions for using the application:

- Access: Available via a button in the navigation bar, opening a sidebar.
- Content:
 - Getting Started: Instructions for typing Dzongkha text and selecting suggested words.
 - Keyboard Toggle: Guide to turning the virtual keyboard on and off.
 - Importing Text: Steps for importing text from DOCX or TXT files.
 - Sharing Text: Instructions for sharing text using the share button.
 - Exporting Text: Steps to export text in different formats.
 - Additional Information: Surfing to the "About Us" page and returning to the main interface.

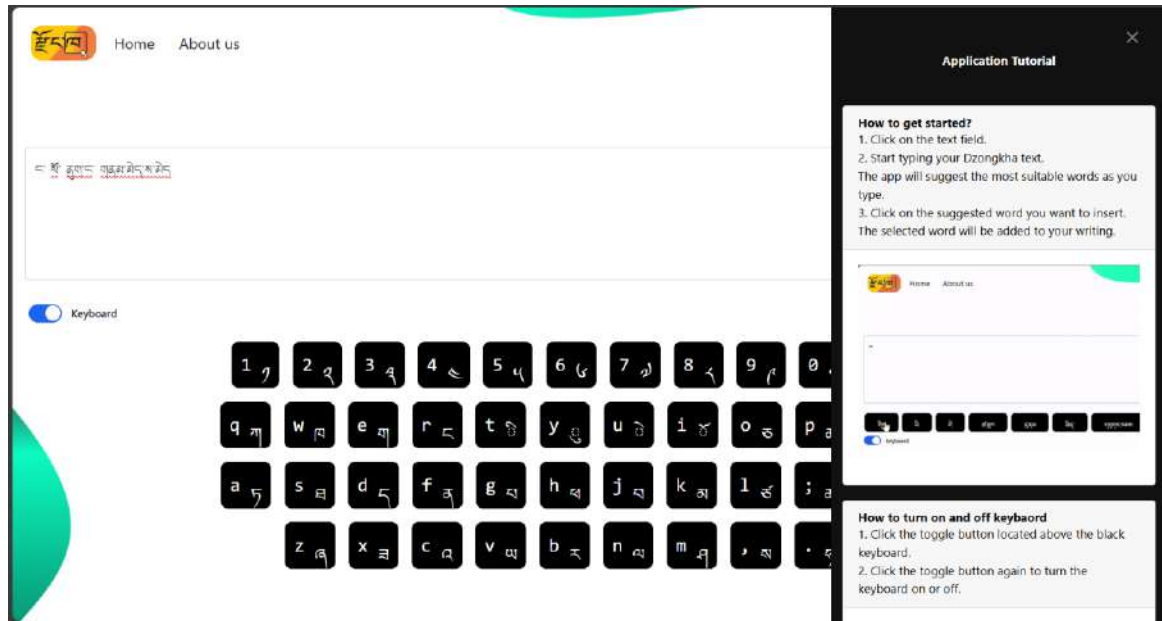


Figure 33. Application Tutorial Section

8.4 Backend Functionality

Below are the insides of the application's backend, which is all powered by Django and posted using a machine learning model to get the predictions:

1. Model Integration:

- The model was developed through TensorFlow and is then loaded within the Django backend in a way that makes it suitable for making predictions.
- The model uses current text input to predict the next word using the tokenizer and sequence padding.

2. Data Management:

- User inputs and predictions are stored in an SQLite database for tracking and analysis.
- The backend will also take care of the file uploads in imported text and generate proper responses for the frontend.

8.5 Implementation Details

The implementation of the use case followed a few simple steps:

1. Setting Up the Django Project:

- Establishing project structure and settings.

- Configure the URLs and views for handling different pages and functionalities.
2. Frontend Development:
 - UI design with HTML, CSS, and Bootstrap.
 - Writing JavaScript functions to process user interactions and AJAX calls.
 3. Model Training and Integration:
 - Create a large dataset of Dzongkha text to be used in training the machine learning model.
 - Loading the trained model in Django and integrating it with the text input field for real-time predictions.
 4. Database Management:
 - Setting up the SQLite database and defining models for storing user inputs and predictions.
 - Implementing views and forms for handling data operations.

8.6 Challenges and Solutions

The following drawbacks were observed during the development of the web application:

1. Data Preprocessing:
 - Handling the unique characteristics of Dzongkha text required extensive preprocessing to ensure accurate tokenization and predictions.
2. Model Accuracy:
 - Ensuring the model provided accurate predictions required fine-tuning the hyperparameters and experimenting with different architectures (LSTM, GRU).
3. User Interface:
 - Designing a user-friendly interface that supports Dzongkha characters and integrates seamlessly with the backend predictions was challenging but essential for user adoption.
 - Implementing caching strategies to reduce the load on the server.

Chapter 9: Result analysis and discussion

9.1. Model Comparison for 20K dataset.

Firstly we trained each model in English and then we implemented it in Dzongkha. Each model was trained using the dataset that we mentioned earlier with the hyperparameters such as Epoch, Layers, Batch Size and Dropout.

For the 20000 sentence dataset.

Sl.No	Model Name	Epochs	Time	Layers	Batch Size	Dropout	Accuracy(%)
1	LSTM	200	1hr	200	64	0.2	73.79
2	Bi-LSTM	200	30mins	200	64	0.2	71.14
3	GRU	200	17mins	200	64	0.2	70.04

Table 4 : Model Comparison on 20k dataset

We achieved the highest accuracy of 73.79% with the LSTM model.

9.2 Graphical Representation of Model performance for 20K dataset.

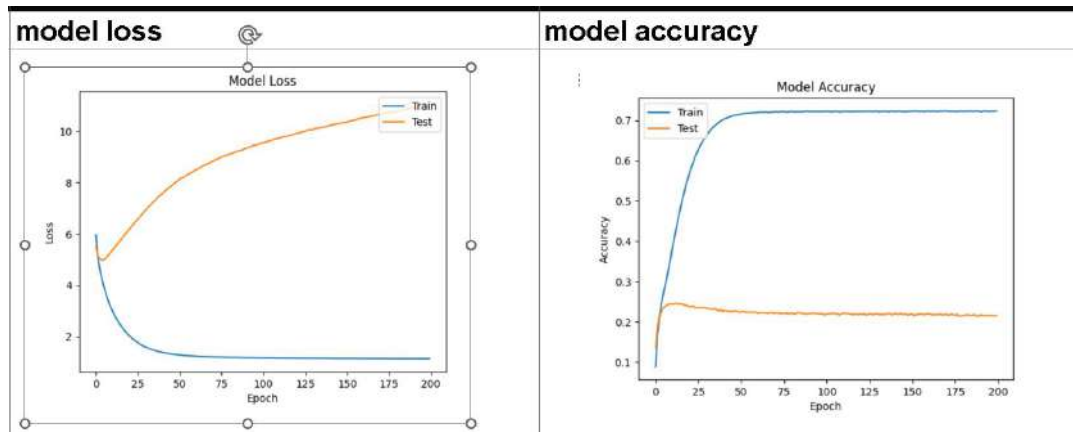


Figure 34. LSTM (20K dataset) result in graphical form

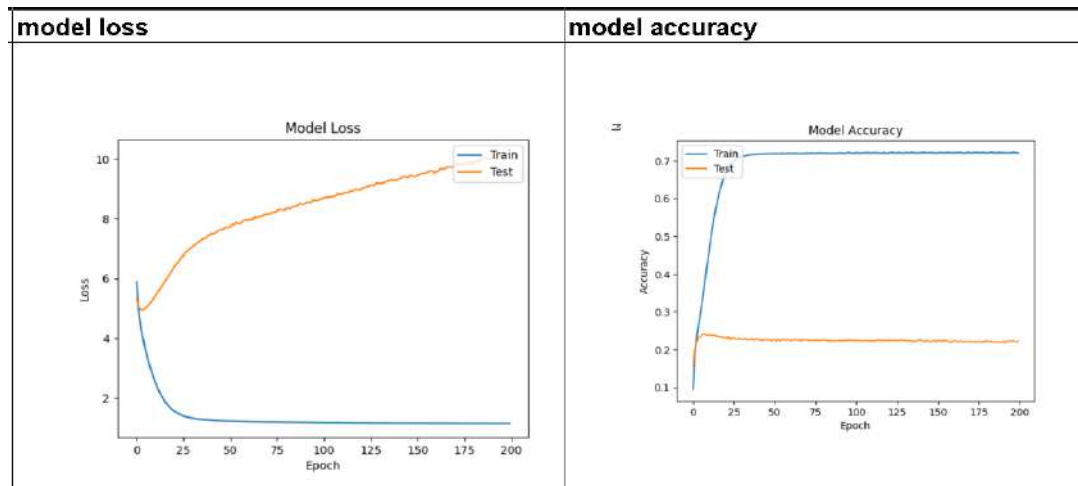


Figure 35. Bi-LSTM (20K dataset) result in graphical form

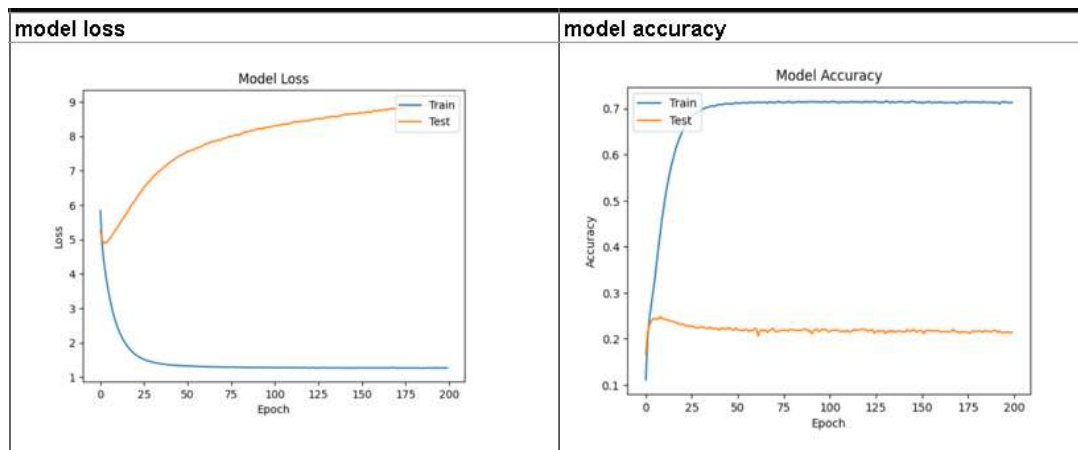


Figure 36. GRU (20K dataset) results in graphical form.

Although we obtained a higher value for accuracy, we observed the issue of overfitting. The models were not able to make correct predictions for the new data samples even though they made accurate predictions for the samples that were used for training. It can also be visually observed in the graph that the gap between the training and the test curve was quite large.

9.3 Model Comparison for 100K dataset.

The size of the dataset was increased.

SL. No	Variable	Value
1	Total sentence	100,000
2	Total words	1,331,282
3	Total no. of unique words	28,344
4	No. of sequence	623,820

Table 5: 100k Dataset Details

The figure below shows the model performance while training it on 100K dataset. The hyperparameters such as Epoch, Dense Layers, Model Layers, neuron, Batch Size, Early Stopping and Dropout were used.

Sl. No	Model Name	Data set	Dense Layers	Model Layers	neuron	Epoch	Time (hours)	Batch Size	Dropout	Early Stopping	Accuracy (%)
1	LSTM	100k	2	2	1024, 512	200	8	500	0.2	10	60
2	BiLSTM	100k	2	2	1024, 512	200	13	500	0.2	10	70.79
3	GRU	100k	2	2	1024, 512	200	15.33	500	0.2	10	74.03

Table 6: Model comparison on 100k dataset

We achieved the highest accuracy of 74.03% with the GRU model.

9.4 Graphical Representation of Model performance for 100K dataset.

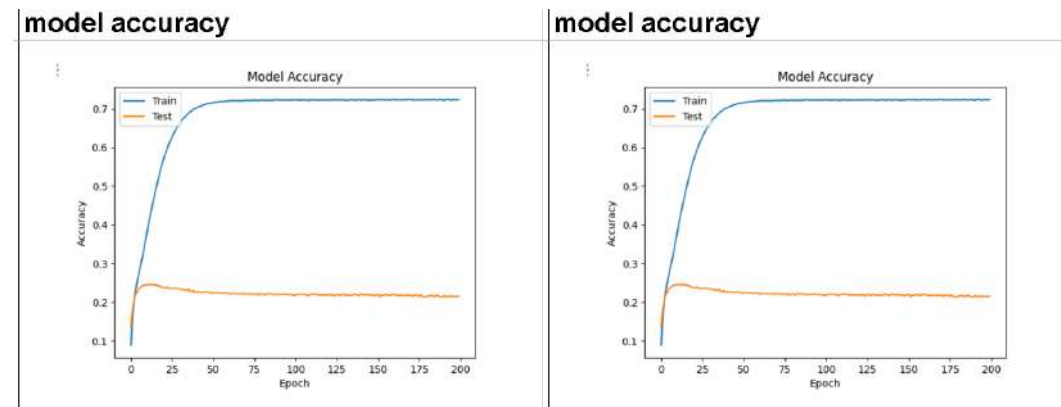


Figure 37. LSTM (100K) result in graphical form

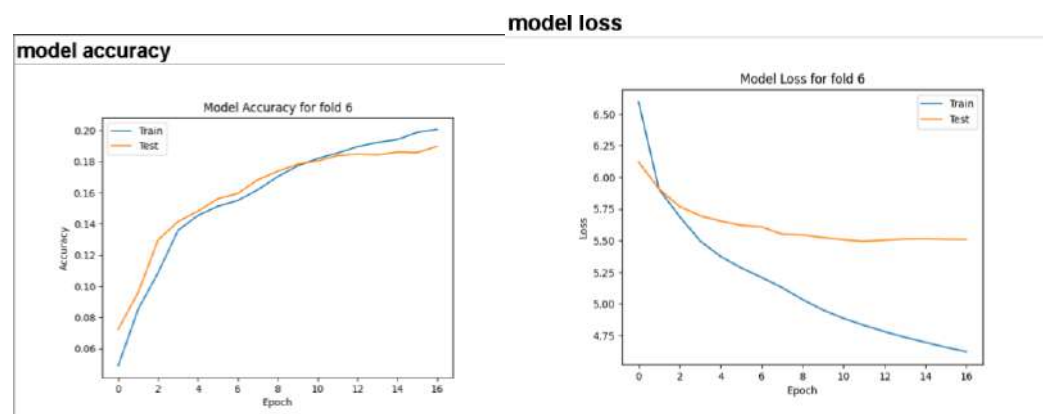


Figure 38. GRU (100K) result in graphical form

Acceptable level of accuracy was obtained while training the model on the dataset listed above but it totally solved the problem of overfitting, the model was able to make accurate predictions for the samples that were not only used for training but also for the new data samples. GRU model is used for integration with the Dzongkha Next Word Prediction Application.

Conclusion

In a nutshell, Our project successfully implemented the GRU model architecture for Next word prediction and achieved promising results. The BERT model had a limitation whereby it marked the next word to be predicted as [MASK]. Initially, due to lack of powerful systems for model training, we trained the model on a smaller dataset and all the models gave us an acceptable level of accuracy such as 73.79% for LSTM, 71.14 for Bi-LSTM and 70.04 for GRU but we faced issues of overfitting. During the second time, we were equipped with GPU system for training our model, we introduced some new hyperparameters and the model hyperparameters underwent some fine tuning and the percentage of accuracy obtained were 60% for LSTM, 70.79 % for Bi-LSTM and 74.03% for GRU and it totally solved the problem of overfitting. Thus, GRU was used for integration with our front-end system. Some of our future works includes development of software package(.exe) for our web app and further improvement of the model to bring about better overall performance of our app.

References

- Avasthi, S., Chauhan, R., & Acharjya, D. P. (2021). Processing large text corpus using N-Gram language modeling and smoothing. In *Lecture notes in networks and systems* (pp. 21–32). https://doi.org/10.1007/978-981-15-9689-6_3
- Bangla Word Prediction and Sentence Completion Using GRU: An Extended Version of RNN on N-gram Language Model. (2019). *IEEE Conference Publication | IEEE Xplore*. <https://ieeexplore.ieee.org/document/9068063>
- Barman, P. P., & Boruah, A. (2018). A RNN based Approach for next word prediction in Assamese Phonetic Transcription. *Procedia Computer Science*, 143, 117–123. <https://doi.org/10.1016/j.procs.2018.10.359>
- Chen, M. (2019, October 8). *Federated learning of N-Gram language Models*. arXiv.org. <https://arxiv.org/abs/1910.03432>
- Endalie, D., Haile, G., & Taye, W. (2022). Bi-directional long short term memory-gated recurrent unit model for Amharic next word prediction. *PLOS ONE*, 17(8), e0273156. <https://doi.org/10.1371/journal.pone.0273156>
- Ezen-Can, A. (2020, September 11). *A comparison of LSTM and BERT for small corpus*. arXiv.org. <https://arxiv.org/abs/2009.05451>
- Habib, M., Faris, M., Qaddoura, R., Alomari, A., & Faris, H. (2021). A Predictive text System for Medical Recommendations in Telemedicine: A Deep Learning approach in the Arabic context. *IEEE Access*, 9, 85690–85708. <https://doi.org/10.1109/access.2021.3087593>
- Hamarashid, H. K., Saeed, S., & Rashid, T. A. (2020). Next word prediction based on the N-gram model for Kurdish Sorani and Kurmanji. *Neural Computing and Applications*, 33(9), 4547–4566. <https://doi.org/10.1007/s00521-020-05245-3>
- Prajapati, G. L., & Saha, R. (2019). REEDS: Relevance and enhanced entropy based

- Dempster Shafer approach for next word prediction using language model.
Journal of Computational Science, 35, 1–11.
<https://doi.org/10.1016/j.jocs.2019.05.001>
- Shakhovska, K., Dumyn, I., Kryvinska, N., & Kagita, M. K. (2021). An approach for a Next-Word prediction for Ukrainian language. *Wireless Communications and Mobile Computing*, 2021, 1–9. <https://doi.org/10.1155/2021/5886119>
- Sharma, M. K., & Samanta, D. (2014). Word prediction system for text entry in Hindi. *ACM Transactions on Asian Language Information Processing*, 13(2), 1–29.
<https://doi.org/10.1145/2617590>
- The performance of LSTM and BILSTM in forecasting time series*. (2019, December 1).
IEEE Conference Publication | IEEE Xplore.
<https://ieeexplore.ieee.org/document/9005997>
- Wangchuk, K., Riyamongkol, P., & Waranusast, R. (2021). Next syllables prediction system in Dzongkha using long short-term memory. *Journal of King Saud University - Computer and Information Sciences*.
<https://doi.org/10.1016/j.jksuci.2021.01.001>
- Wangchuk, K., Wangchuk, T., & Namgyel, T. (2023b). Dzongkha Next Words Prediction Using Bidirectional LSTM. *Bhutan Journal of Research and Development*, 2.
<https://doi.org/10.17102/bjrd.rub.se2.038>
- Word Completion and Sequence Prediction in Bangla Language Using Trie and a Hybrid Approach of Sequential LSTM and N-gram. (2020). *IEEE Conference Publication* | *IEEE Xplore*. <https://ieeexplore.ieee.org/document/9333518>